

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 15

TITLE:

Design and develop an Automated Color-Based Product Sorting System for Manufacturing Industries. The system automatically detects and sorts products based on color for industrial automation applications.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

The Automated Color-Based Product Sorting System is designed to automatically detect and sort products according to their color in manufacturing industries. The system uses a color sensor, microcontroller, conveyor mechanism, and sorting actuator to identify the color of products and direct them to their respective collection areas. The color sensor detects the color of each product moving on the conveyor. The microcontroller processes the sensor output and determines the corresponding color. Based on the detected color, the controller operates a servo motor or other actuator to sort the product into the appropriate section.

PROBLEM STATEMENT

- Manual identification and sorting of products.
- Time-consuming sorting operations.
- Human errors during color identification.
- Reduced production efficiency.
- Difficulty in continuously sorting large numbers of products.
- Increased dependence on manual labor.
- Lack of automatic product classification.
- Need for faster and more consistent industrial sorting.

PROPOSED SOLUTION

The proposed system uses a color sensor, microcontroller, conveyor belt, and sorting mechanism to automatically classify products based on color.

The system can:

- Detect product presence.
- Identify product color.
- Process the detected color.
- Control the conveyor movement.
- Activate the appropriate sorting mechanism.
- Direct products to different collection sections.
- Display sorting status.
- Reduce manual intervention.

The controller makes the sorting decision according to the color detected by the sensor.

WORKING PRINCIPLE

- The product is placed on the conveyor belt.
- The conveyor moves the product toward the color sensor.
- The sensor detects the color of the product.
- The sensor sends the color information to the microcontroller.
- The microcontroller identifies the corresponding color.
- The controller determines the appropriate sorting position.
- The conveyor moves the product toward the sorting point.
- The actuator moves to the required position.
- The product is directed into the corresponding collection bin.
- The actuator returns to its initial position.
- The next product is detected and sorted.

SYSTEM ARCHITECTURE

Product Input → Conveyor Belt → Color Sensor → Microcontroller → Motor/Servo Driver → Sorting Mechanism → Collection Bins

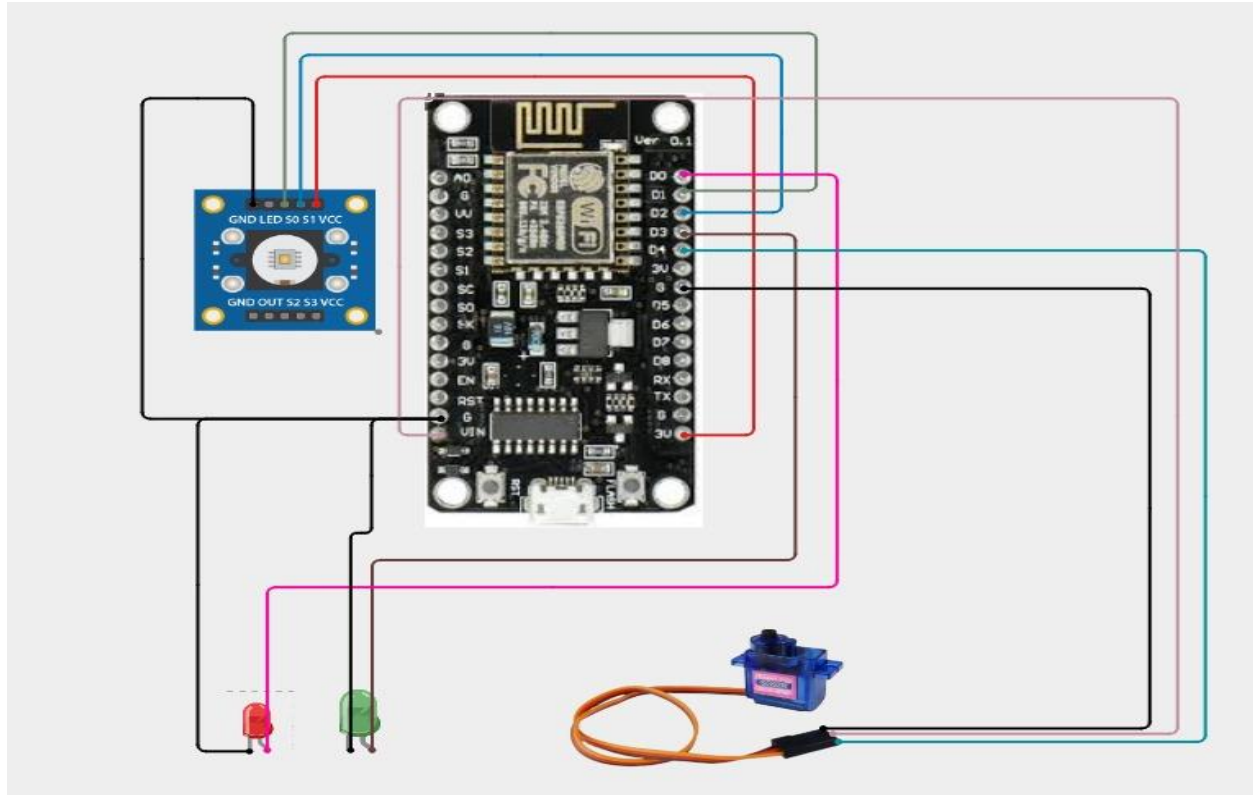
PRODUCT SORTING OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Product Detection	Product detected	Intermittent detection	No detection
Color Detection	Correct color	Uncertain color	Sensor failure
Conveyor	Normal operation	Delay	Motor fault
Sorting Mechanism	Correct position	Position delay	Actuator fault
Product Flow	Continuous	Slow	Blocked
System Status	Sorting	Warning	System stopped

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Product Presence	IR/Proximity Sensor	Detect product
Product Color	Color Sensor	Identify product color
Conveyor Status	DC Motor/Motor Driver	Control product movement
Sorting Position	Servo Motor/Actuator	Direct product
System Status	LED/Display	Show operating condition
Sorting Status	Microcontroller	Confirm sorting operation

CIRCUIT DESIGN



ALGORITHM

START

- Initialize the microcontroller.
- Initialize the color sensor.
- Initialize the product-detection sensor.
- Initialize the conveyor motor.
- Initialize the sorting actuator.
- Initialize the display/LED indicators.
- Start the conveyor.
- Detect the presence of a product.
- Stop or position the product in front of the color sensor.
- Read the color sensor values.
- Identify the product color.
- Compare the detected color with predefined color values.
- Determine the corresponding sorting position.
- Move the conveyor toward the sorting section.
- Activate the sorting actuator.

- Direct the product into the appropriate bin.
- Return the actuator to its initial position.
- Restart the conveyor.
- Repeat the process for the next product.
- **STOP/REPEAT**

CODING

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <Servo.h>

// =====
// WI-FI DETAILS
// =====
#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO DETAILS
// =====
#define IO_USERNAME  "Hair_13"
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);

// =====
// TCS3200 CONNECTIONS
// =====
#define S0 D1
#define S1 D2
#define S2 D5
#define S3 D6
#define SENSOR_OUT D7

// =====
// SERVO
// =====
#define SERVO_PIN D4

Servo sorterServo;

// =====
```

```

// LEDS
// =====
#define LED1 D0
#define LED2 D3

// =====
// ADAFRUIT IO FEEDS
// =====
AdafruitIO_Feed *colorFeed = io.feed("color");
AdafruitIO_Feed *servoFeed = io.feed("servo");
AdafruitIO_Feed *redFeed = io.feed("red");
AdafruitIO_Feed *greenFeed = io.feed("green");
AdafruitIO_Feed *blueFeed = io.feed("blue");

// =====
// COLOR SENSOR FUNCTION
// =====
int readColor(bool s2State, bool s3State)
{
    digitalWrite(S2, s2State);
    digitalWrite(S3, s3State);

    delay(100);

    return pulseIn(SENSOR_OUT, LOW, 100000);
}

// =====
// SETUP
// =====
void setup()
{
    Serial.begin(115200);

    // ----- TCS3200 -----
    pinMode(S0, OUTPUT);
    pinMode(S1, OUTPUT);
    pinMode(S2, OUTPUT);
    pinMode(S3, OUTPUT);
    pinMode(SENSOR_OUT, INPUT);

    // 20% frequency scaling
    digitalWrite(S0, HIGH);

```

```

digitalWrite(S1, LOW);

// ----- LEDs -----
pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);

digitalWrite(LED1, LOW);
digitalWrite(LED2, LOW);

// ----- Servo -----
sorterServo.attach(SERVO_PIN);

// Start position
sorterServo.write(90);

// ----- Adafruit IO -----
Serial.println();
Serial.println("=====");
Serial.println("COLOR SORTING SYSTEM");
Serial.println("=====");

Serial.println("Connecting to Adafruit IO...");

io.connect();

while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");
Serial.println();
}

// =====
// LOOP
// =====
void loop()
{
    // Keep Adafruit IO connected

```

```
io.run();

// =====
// READ COLOR VALUES
// =====

int redValue = readColor(LOW, LOW);
int greenValue = readColor(HIGH, HIGH);
int blueValue = readColor(LOW, HIGH);

// =====
// SERIAL MONITOR
// =====

Serial.println("-----");

Serial.print("RED  = ");
Serial.println(redValue);

Serial.print("GREEN = ");
Serial.println(greenValue);

Serial.print("BLUE  = ");
Serial.println(blueValue);

// =====
// SEND RGB VALUES TO ADAFRUIT
// =====

redFeed->save(redValue);
greenFeed->save(greenValue);
blueFeed->save(blueValue);

// Turn both LEDs OFF
digitalWrite(LED1, LOW);
digitalWrite(LED2, LOW);

// =====
// RED DETECTION
// =====
```

```
if (redValue < greenValue &&
    redValue < blueValue &&
    redValue > 0)
{
    Serial.println("COLOR DETECTED = RED");

    digitalWrite(LED1, HIGH);

    sorterServo.write(30);

    colorFeed->save("RED");
    servoFeed->save(30);

    delay(1000);
}

// =====
// GREEN DETECTION
// =====

else if (greenValue < redValue &&
        greenValue < blueValue &&
        greenValue > 0)
{
    Serial.println("COLOR DETECTED = GREEN");

    digitalWrite(LED2, HIGH);

    sorterServo.write(90);

    colorFeed->save("GREEN");
    servoFeed->save(90);

    delay(1000);
}

// =====
// BLUE DETECTION
// =====

else if (blueValue < redValue &&
        blueValue < greenValue &&
        blueValue > 0)
{
```

```

    Serial.println("COLOR DETECTED = BLUE");

    sorterServo.write(150);

    colorFeed->save("BLUE");
    servoFeed->save(150);

    delay(1000);
}

// =====
// UNKNOWN COLOR
// =====

else
{
    Serial.println("COLOR DETECTED = UNKNOWN");

    sorterServo.write(90);

    colorFeed->save("UNKNOWN");
    servoFeed->save(90);
}

Serial.println("-----");

delay(1000);
}

```

SYSTEM WORKING

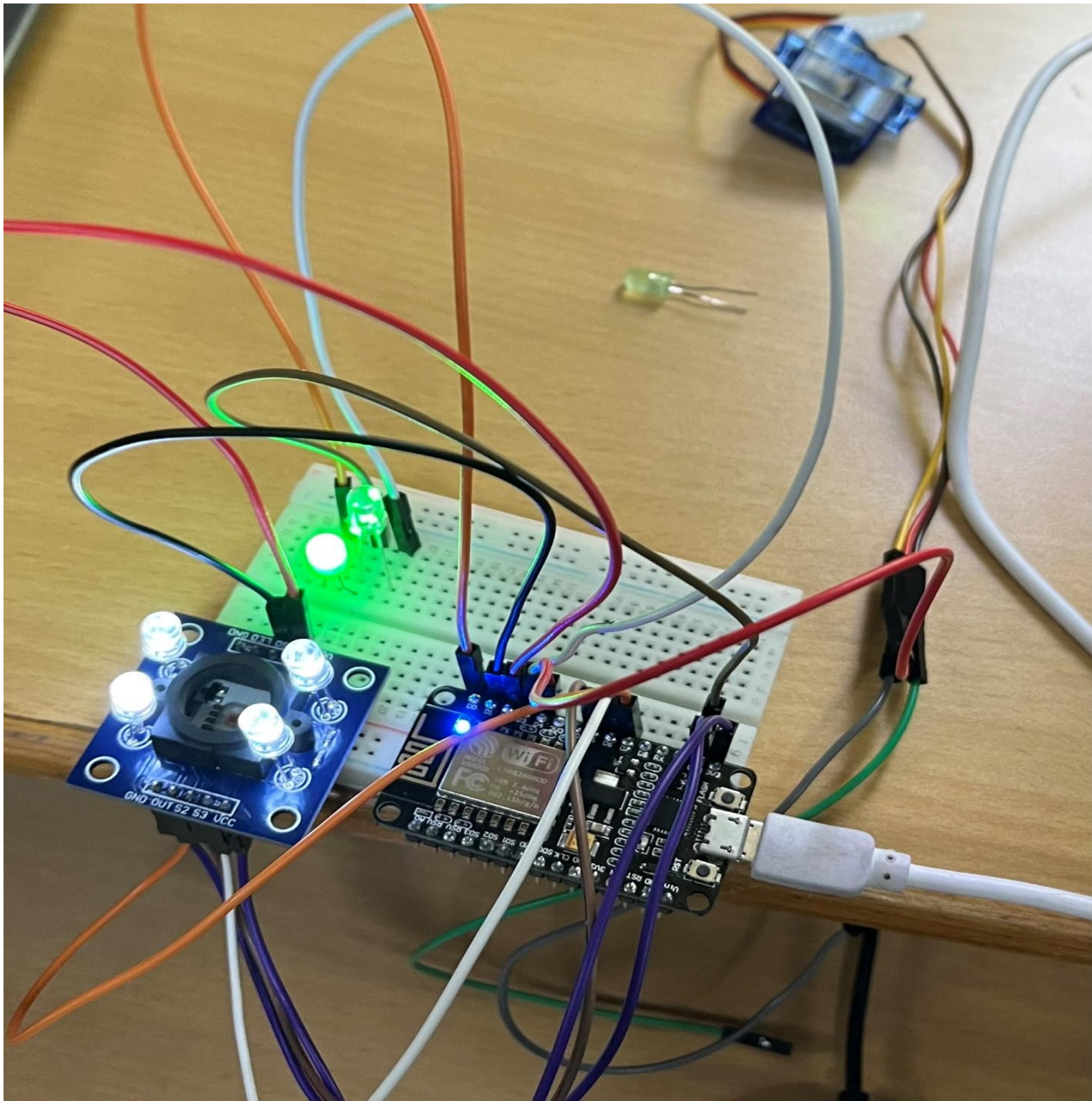
The system begins by moving products along a conveyor belt. A product-detection sensor identifies when a product reaches the color-sensing area. The conveyor positions the product in front of the color sensor. The color sensor measures the reflected light from the product and provides the information to the microcontroller. The microcontroller compares the sensor readings with predefined color conditions and identifies the product color. After identification, the conveyor transports the product toward the sorting point. A servo motor or actuator moves the sorting mechanism to the corresponding position and directs the product into its assigned collection bin. The sorting mechanism then returns to its initial position, and the conveyor continues to transport the next product.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	Arduino/ESP32 Microcontroller	1	Main controller
2	Color Sensor	1	Detect product color
3	IR/Proximity Sensor	1	Detect product presence
4	DC Gear Motor	1	Conveyor movement
5	Motor Driver	1	Control conveyor motor
6	Servo Motor	1 or more	Product sorting
7	Conveyor Belt	1	Transport products
8	Collection Bins	As required	Collect sorted products
9	LCD/OLED Display	1	Display system status
10	LEDs	As required	Status indication
11	Buzzer	1	Warning indication
12	Power Supply	1	Power source
13	Breadboard & Jumper Wires	1 set	Circuit connections

PROTOTYPE IMPLEMENTATION

The prototype can be developed using an Arduino/ESP32 microcontroller, color sensor, IR sensor, conveyor belt, DC motor, motor driver, and servo motor. The color sensor is positioned above the conveyor at the product detection point. When a product reaches this position, the sensor detects its color. The microcontroller processes the sensor output and determines the appropriate sorting category. The conveyor motor transports the product toward the sorting location. A servo motor or mechanical actuator is used to divert the product into the appropriate collection bin. Separate bins can be provided for different colors. An LCD/OLED display and LEDs can be used to indicate the detected color and system status. The prototype can be implemented or simulated using suitable platforms such as Arduino, Proteus, or Tinkercad, depending on the required project setup.



RESULTS & PERFORMANCE ANALYSIS

Condition	Color Sensor Output	System Action
Product detected	Valid detection	Color sensing begins
Red product	Red identified	Product directed to red bin
Blue product	Blue identified	Product directed to blue bin
Green product	Green identified	Product directed to green bin
Unknown color	No matching color	Warning/rejection action
No product	No detection	Conveyor continues/standby
Sorting completed	Valid	Actuator returns to initial position

PERFORMANCE

- Automatic product detection
- Automatic color identification
- Automatic product sorting
- Reduced manual intervention
- Improved sorting efficiency
- Reduced human error
- Continuous product processing
- Automatic conveyor control
- Servo-based sorting mechanism
- Real-time sorting status
- Improved manufacturing automation
- Better consistency in product classification
- Low-cost prototype suitable for educational and industrial automation applications

